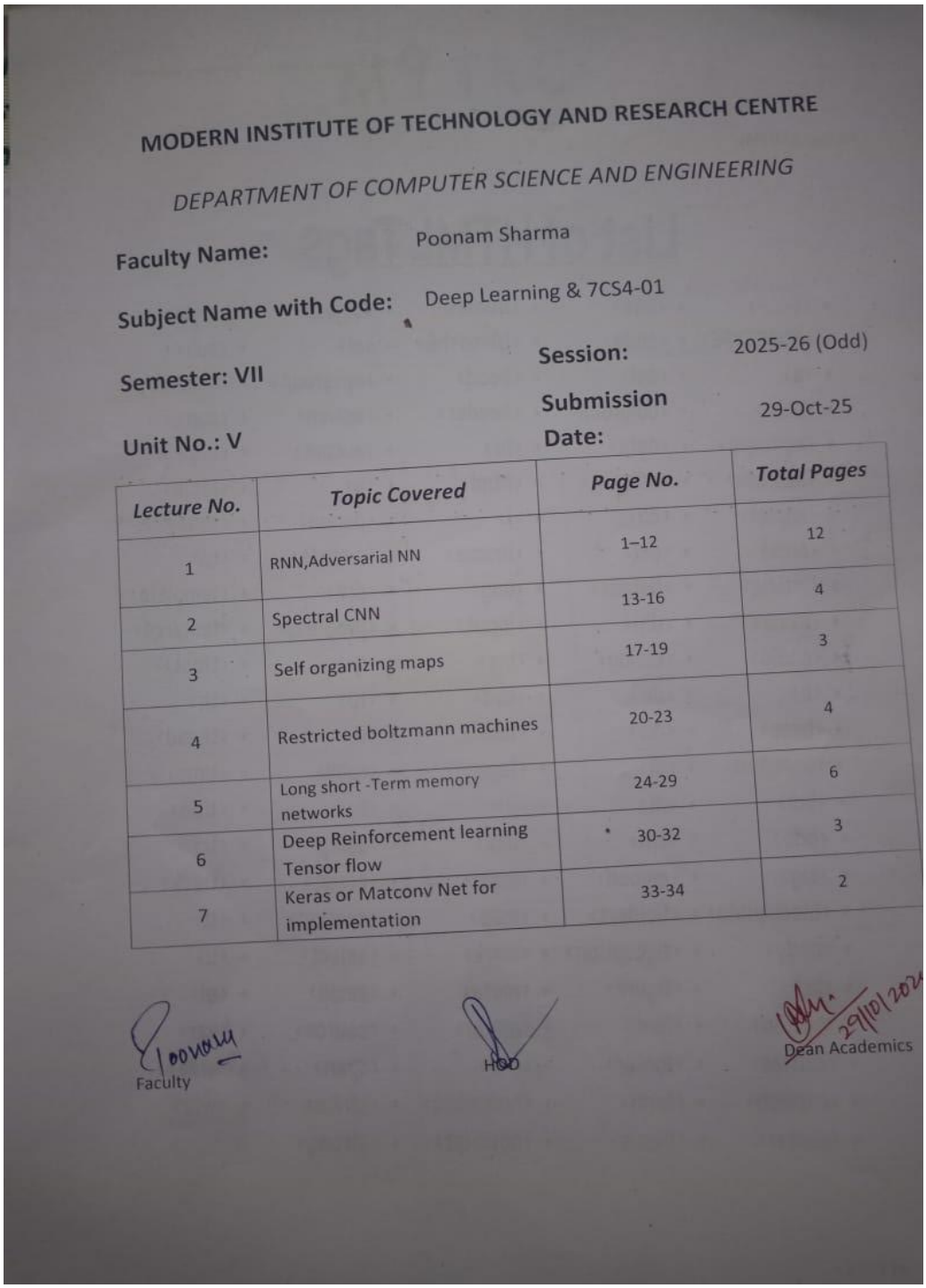


MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty Name: Poonam Sharma

Subject Name with Code: Deep Learning & 7CS4-01

Semester: VII **Session:** 2025-26 (Odd)

Unit No.: V **Submission Date:** 29-Oct-25

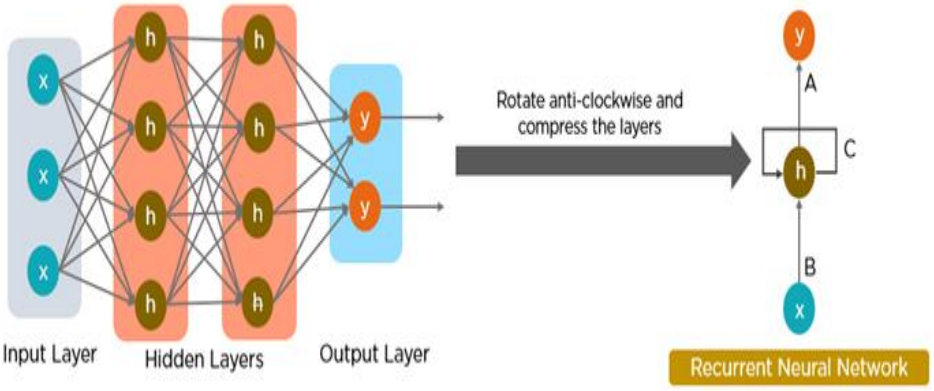
Lecture No.	*Topic Covered*	*Page No.*	*Total Pages*
1	RNN,Adversarial NN	1–12	12
2	Spectral CNN	13-16	4
3	Self organizing maps	17-19	3
4	Restricted boltzmann machines	20-23	4
5	Long short -Term memory networks	24-29	6
6	Deep Reinforcement learning Tensor flow	30-32	3
7	Keras or Matconv Net for implementation	33-34	2

Faculty HOD Dean Academics

UNIT-V
Neural Network Architectures
Lecture No.1

Recurrent Neural Networks

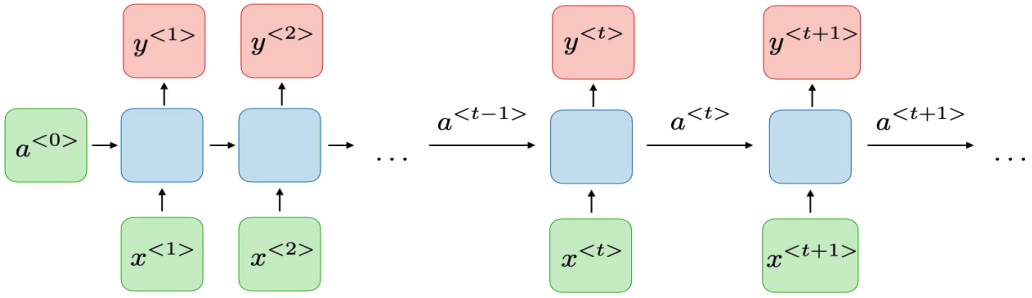
Neural networks imitate the function of the human brain in the fields of AI, machine learning, and deep learning, allowing computer programs to recognize patterns and solve common issues. RNNs are a type of neural network that can be used to model sequence data. RNNs, which are formed from feed forward networks, are similar to human brains in their behaviour. Simply said, recurrent neural networks can anticipate sequential data in a way that other algorithms can't.



All of the inputs and outputs in standard neural networks are independent of one another, however in some circumstances, such as when predicting the next word of a phrase, the prior words are necessary, and so the previous words must be remembered. As a result, RNN was created, which used a Hidden Layer to overcome the problem. The most important component of RNN is the Hidden state, which remembers specific information about a sequence.RNNs have a Memory that stores all information about the calculations. It employs the same settings for each input since it produces the same outcome by performing the same task on all inputs or hidden layers.

The Architecture of a Traditional RNN

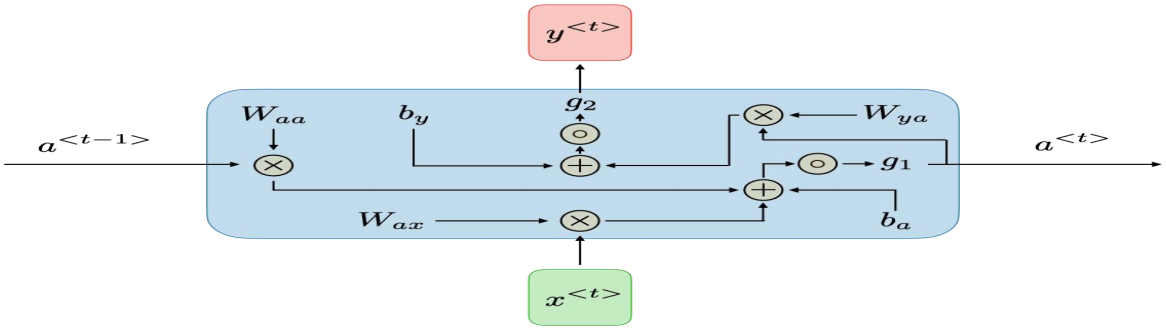
RNNs are a type of neural network that has hidden states and allows past outputs to be used as inputs. They usually go like this:



For each timestep t, the activation $a^{<t>}$ and the output $y^{<t>}$ are expressed as follows:

$$a^{<t>} = g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a) \quad \text{and} \quad y^{<t>} = g_2(W_{ya}a^{<t>} + b_y)$$

where $W_{ax}, W_{aa}, W_{ya}, b_a, b_y$ are coefficients that are shared temporally and g_1, g_2 activation functions.



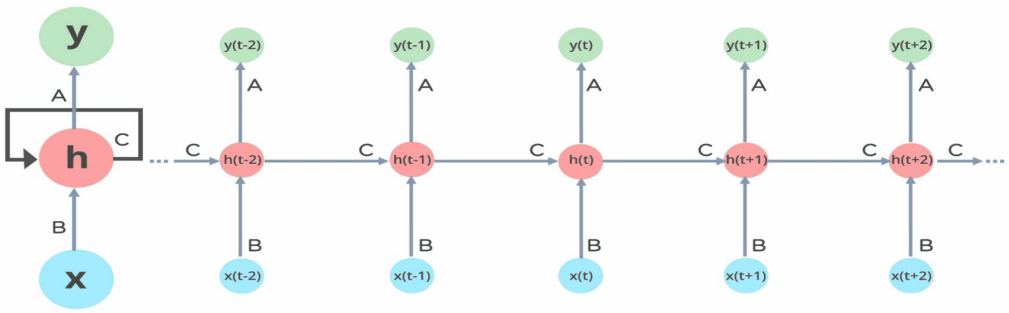
RNN architecture can vary depending on the problem you're trying to solve. From those with a single input and output to those with many (with variations between).

Below are some examples of RNN architectures that can help you better understand this.

- **One To One:** There is only one pair here. A one-to-one architecture is used in traditional neural networks.
- **One To Many:** A single input in a one-to-many network might result in numerous outputs. One too many networks are used in the production of music, for example.
- **Many To One:** In this scenario, a single output is produced by combining many inputs from distinct time steps. Sentiment analysis and emotion identification use such networks, in which the class label is determined by a sequence of words.
- **Many To Many:** For many to many, there are numerous options. Two inputs yield three outputs. Machine translation systems, such as English to French or vice versa translation systems, use many to many networks.

How does Recurrent Neural Networks work?

The information in recurrent neural networks cycles through a loop to the middle hidden layer.

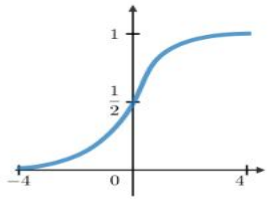
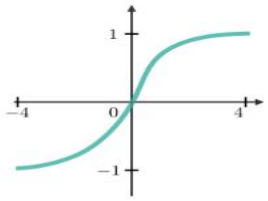
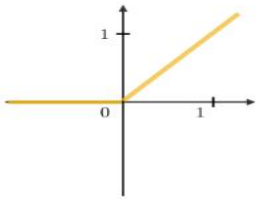


The input layer **x** receives and processes the neural network's input before passing it on to the middle layer.Multiple hidden layers can be found in the middle layer **h**, each with its own activation functions, weights, and biases. You can utilize a recurrent neural network if the various parameters of different hidden layers are not impacted by the preceding layer, i.e. there is no memory in the neural network.

The different activation functions, weights, and biases will be standardized by the Recurrent Neural Network, ensuring that each hidden layer has the same characteristics. Rather than constructing numerous hidden layers, it will create only one and loop over it as many times as necessary.

Common Activation Functions

A neuron's activation function dictates whether it should be turned on or off. Nonlinear functions usually transform a neuron's output to a number between 0 and 1 or -1 and 1.

Sigmoid	Tanh	RELU
$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$
		

The following are some of the most commonly utilized functions:

- **Sigmoid:** The formula **g(z) = 1/(1 + e^-z)** is used to express this.
- **Tanh:** The formula **g(z) = (e^-z – e^-z)/(e^-z + e^-z)** is used to express this.
- **Relu:** The formula **g(z) = max(0 , z)** is used to express this.

Advantages of RNNs:

- Handle sequential data effectively, including text, speech, and time series.
- Process inputs of any length, unlike feedforward neural networks.

- Share weights across time steps, enhancing training efficiency.

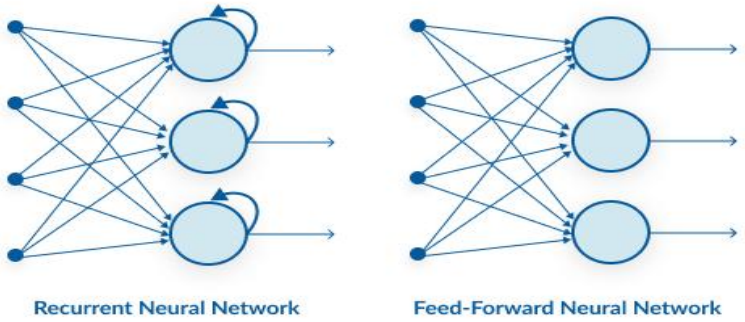
Disadvantages of RNNs:

- Prone to vanishing and exploding gradient problems, hindering learning.
- Training can be challenging, especially for long sequences.
- Computationally slower than other neural network architectures.

Recurrent Neural Network Vs Feed forward Neural Network

A feed-forward neural network has only one route of information flow: from the input layer to the output layer, passing through the hidden layers. The data flows across the network in a straight route, never going through the same node twice.

The information flow between an RNN and a feed-forward neural network is depicted in the two figures below.



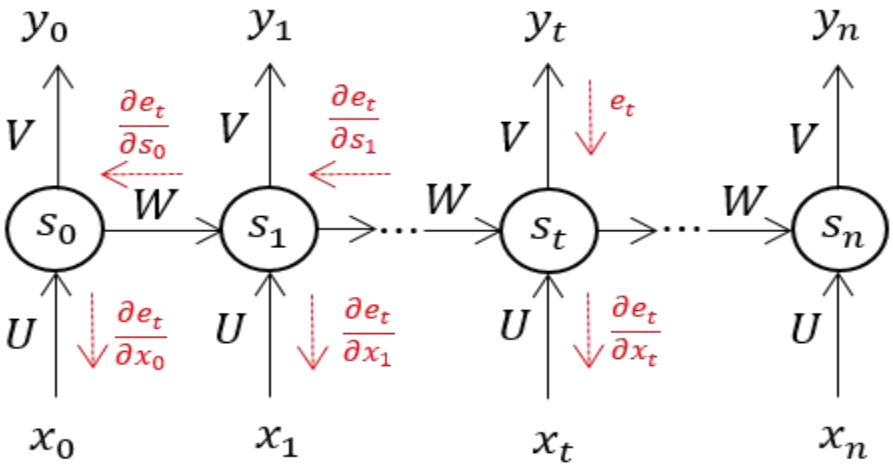
Feed-forward neural networks are poor predictions of what will happen next because they have no memory of the information they receive. Because it simply analyses the current input, a feed-forward network has no idea of temporal order. Apart from its training, it has no memory of what transpired in the past.

The information is in an RNN cycle via a loop. Before making a judgment, it evaluates the current input as well as what it has learned from past inputs. A recurrent neural network, on the other hand, may recall due to internal memory. It produces output, copies it, and then returns it to the network.

Backpropagation Through Time (BPTT)

When we apply a Backpropagation algorithm to a Recurrent Neural Network with time series data as its input, we call it backpropagation through time.

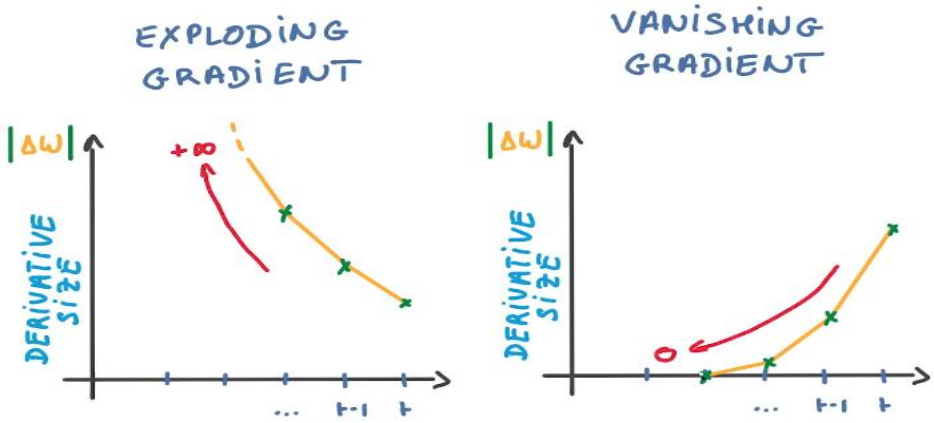
A single input is sent into the network at a time in a normal RNN, and a single output is obtained. Backpropagation, on the other hand, uses both the current and prior inputs as input. This is referred to as a timestep, and one timestep will consist of multiple time series data points entering the RNN at the same time.



The output of the neural network is used to calculate and collect the errors once it has trained on a time set and given you an output. The network is then rolled back up, and weights are recalculated and adjusted to account for the faults.

Two issues of Standard RNNs

There are two key challenges that RNNs have had to overcome, but in order to comprehend them, one must first grasp what a gradient is.



With regard to its inputs, a gradient is a partial derivative. If you're not sure what that implies, consider this: a gradient quantifies how much the output of a function varies when the inputs are changed slightly.

A function's slope is also known as its gradient. The steeper the slope, the faster a model can learn, the higher the gradient. The model, on the other hand, will stop learning if the slope is zero. A gradient is used to measure the change in all weights in relation to the change in error.

- **Exploding Gradients:** Exploding gradients occur when the algorithm gives the weights an absurdly high priority for no apparent reason. Fortunately, truncating or squashing the gradients is a simple solution to this problem.
- **Vanishing Gradients:** Vanishing gradients occur when the gradient values are too small, causing the model to stop learning or take far too long. This was a big issue in the 1990s, and it was far more difficult to address than the exploding gradients. Fortunately, Sepp Hochreiter and Juergen Schmidhuber's LSTM concept solved the problem.

RNN Applications

Recurrent Neural Networks are used to tackle a variety of problems involving sequence data. There are many different types of sequence data, but the following are the most common: Audio, Text, Video, Biological sequences.

Using RNN models and sequence datasets, you may tackle a variety of problems, including :

- Speech recognition
- Generation of music
- Automated Translations
- Analysis of video action
- Sequence study of the genome and DNA.

References

1. https://www.geeksforgeeks.org/introduction-to-recurrent-neural-network/

UNIT-V (Neural Network Architectures)
Adversarial NN
Lecture No.2

Generative Adversarial Networks (GANs) in Machine Learning

A Generative Adversarial Network or GAN is defined as the technique of generative modeling used to generate new data sets based on training data sets. The newly generated data set appears similar to the training data sets.

GANs mainly contain two neural networks capable of capturing, copying, and analyzing the variations in a dataset. These two neural networks are known as generators and discriminators, and both of them compete with each other.

The term Generative Adversarial Network consists of three words, and each of them has its separate meaning, which is as follows:

- **Generative:** It is used to learn a generative model that visually explains how data is generated.
- **Adversarial:** As both neural networks compete with each other or are adversarial to one another, hence training of the model is done in an adversarial manner.
- **Networks:** It uses deep neural networks to train models, hence called networks.

Why need GANs?

With the advancement of technology, there are various possible ways by which ML algorithms and neural networks can be fooled into misclassifying data just by introducing some noise in the training datasets. But with the innovations in machine learning, various techniques are being developed that can prevent the chances of misclassifying the images. Hence, GANs were discovered to generate new data sets, which look like training data sets and hence can start visualizing new patterns like training data.

Applications of GANs

GANs are a much popular approach in machine learning and have various applications in real-world scenarios. Below are a few most famous applications of generative adversarial networks (GANs), which are as follows:

- Fashion, art, and advertising
- Science
- Video games

- Audio synthesis
- Transfer learning

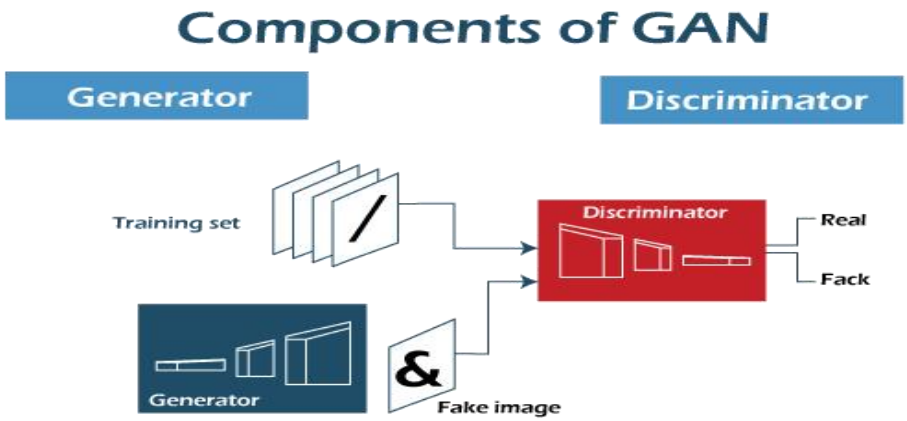
Besides these applications, there are so many miscellaneous applications of GANs in machine learning, which are as follows:

- It is used to diagnose partial or total vision loss by detecting glaucomatous images.
- It is used to visualize the interior design, industrial design, shoes, bags, and clothing items by generating photorealistic images.
- It is used to reconstruct 3D models of objects from images and model motion patterns in the video.
- It is used to develop age face photographs that determine individuals' faces according to their age.
- It is used to denoise welding images by removing the random light reflection on the dynamic weld pool surface.
- It is being used in data augmentation.
- It is used to reconstruct individual faces after listening to their voice. It is known as **GAN Speech2Face technology.**
- It is used to visualize the effects of climate change on particular locations.
- It is used to develop intelligent games and animations by creating anime characters.
- GANs generate text, articles, songs, poems, etc.

As soon as research on GANs in machine learning is going at its peak, in the future, we will see GAN applications in producing high-quality video, audio, and images also. Further, Microsoft has already collaborated with OpenAI to work on GPT and explore the power of GAN at the next level.

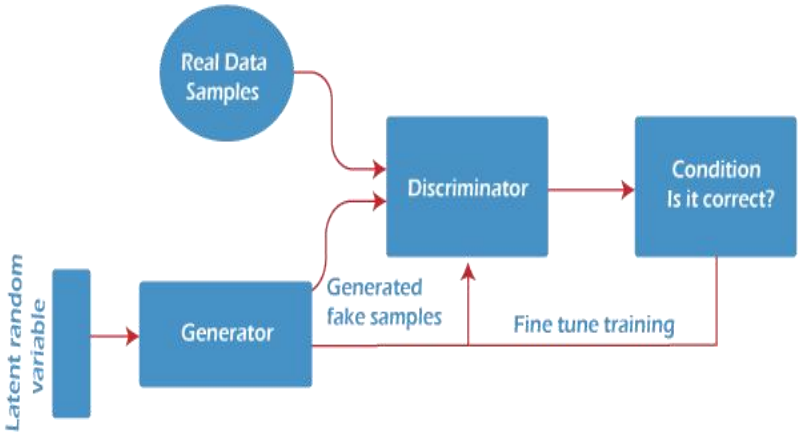
Components of Generative Adversarial Networks (GANs)

Generative adversarial networks are primarily made from two components, i.e., **generator** and **discriminator**. As the name suggests, the generator generates a fake output of unseen data based on training data sets and makes the discriminator fool to understand this fake data as accurate. Further, the discriminator acted as a cop and used to discriminate the training data and generated data, identify the abnormalities in the samples created by the generator and classify them as Fake or genuine. However, this entire process continues till the generator wins and ultimately makes a fool of the discriminator on fake data. Components of GANs are as follows:



- **Discriminator:** It is used as a supervised machine learning approach in which a simple classifier is appointed to discriminate between real and fake data. Although, it is trained on actual training data sets and gives feedback to the generator.
- **Generator:** Unlike the discriminator, the generator is an unsupervised machine learning method used to generate fake samples based on actual training data sets. It is also a neural network with hidden layers, activation, and loss function.

  Further, the generator primarily focuses on generating fake data based on feedback given by the discriminator and makes the discriminator fool so that it cannot identify the difference between actual output and generated output by the generator.



This process continues until the generator makes the discriminator fool, and once this is achieved generalized GAN model is created. Here are a few essential steps to train GANs components individually. These are as follows:

- **Step-1: Identify the actual problems:** This is essential in working on real-time projects. If you can identify the actual problems, you can only solve this efficiently. In GANs, whatever you are aiming for, you need to define that, which means What you want to create, like audio, poem, text, or image, is a type of problem.

- **Step-2: Choose appropriate GAN architecture:** Although there are so many architectures of GANs exist, such as DCGAN, Conditional GAN, Unconditional GAN, Least Square GAN, Auxilary Classifier GAN, Dual Video Discriminator, SRGAN, Cycle GAN, and Info GAN, we have to define which type of GAN architecture we are using in our project.
- **Step-3: Give training to discriminator on real data sets:**
  The discriminator is always given training on real data sets, and it only contains a forward path mechanism and does not follow backpropagation in n epochs. Further, it is only provided with actual data having zero noise or fake content. Further, for fake images, the discriminator uses instances created by the generator as negative output. Some actions happen during the discriminator training process.
- **Step-4: Provide training to the generator:** The training process of the generator starts with the introduction of some fake inputs. Initially, we give some fake input to the generator, but later it generates some fake output by adding some random noise. Further, whenever the generator gets trained, discriminators remain inactive, while the generator remains inactive when the discriminator gets trained. While providing training to generator training using any random noise as input, it aims to convert it into meaningful data to provide meaningful output, and the process takes time and runs under many epochs:

  Below are a few simple steps to train the generator on fake input as follows:

  - Provide fake input or noise and get random noise to produce output based on the noise sample.
  - Predict generator output either real or fake using discriminator.
  - Calculate discriminator loss and perform backpropagation.
  - Calculate gradients to update the weights of the generator.
- **Step-5: Provide training to discriminators on fake inputs:** In this step, we pass the samples to discriminators to predict whether the data is real or fake. Further, provide feedback received by decimators to generators again to do modifications in the samples.

How do GANs work?

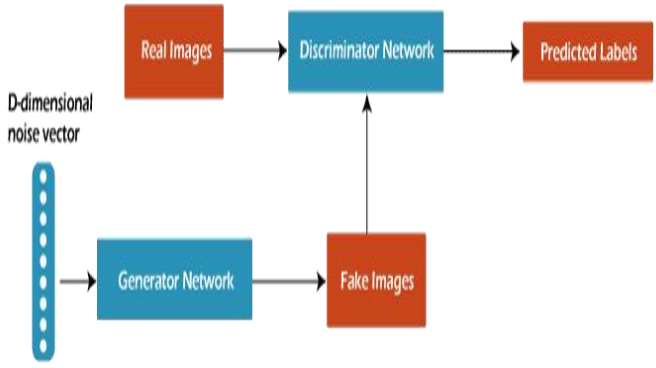
As discussed above, GAN contains two neural networks, of which one is called Generator G(x), and another one is called Discriminator D(x). As the name suggests, both of these works in an Adversarial manner. The generator always tries to generate fake data similar to training data to fool the discriminator, i.e., it generates new data instances. Whereas the discriminator aims to identify the fake data from the actual data, i.e., it evaluates the

authenticity of data. Both neural networks work simultaneously to learn from complex data, including images, audio, or video files.

Let's say we try to generate hand-written numerals similar to as MNIST dataset, which occurred in the real world; now, the discriminator aims to identify the instance of the accurate MNIST dataset as authentic. Meanwhile, the generator creates new synthetic images and passes them to the discriminator. The generator expects these images to be identified as accurate, even if they are fake. It generates as possible hand-written digits to fool the discriminator. The discriminator aims to identify images as fake coming from the generator.

The working of GANs can be summarized in the below steps:

- Firstly, the generator takes in any random number and generates an image.
- The generated image is inputted to the discriminator, and the authentic images are taken from the actual dataset.
- The discriminator contains both real and fake images, and now it aims to predict the labels with the identification of real and fake images. As an output, it returns probabilities of a number between 0 and 1, where 0 represents a prediction of fake and 1 represents authenticity. The working process of GAN is represented below image:



Different types of Generative Adversarial Networks (GAN)s

- **DCGAN:** DCGAN or Deep Convolutional GAN is one of the most famous implementations of GAN. It makes use of ConvNets instead of Multi-layered perceptron. Contents use a convolutional stride and are built without max pooling. Further, layers in ConvNets are not entirely connected.
- **Conditional and Unconditional GAN:** It is defined as a deep learning neural network having extra parameters. In conditional and unconditional GAN, labels are kept in such a way so that they can easily classify the input of the discriminator.

- **Least Square GAN:** It is a particular type of generative adversarial network that uses the least-square loss function for the discriminator. Further, whenever the objective function of least square GAN is minimized, Pearson divergence also gets minimized automatically.
- **Auxiliary Classifier GAN:** ACGAN or Auxiliary Classifier GAN is a similar but improved version of CGAN. Its discriminator not only classifies an image as real or fake but also gives information about the source of the input image.
- **Dual Video Discriminator GAN:** It is the most helpful type of GAN for video generation built upon the BigGAN architecture. Further, it uses a spatial and temporal discriminator for generating videos.
- **SRGAN: Super Resolution or SRGAN** is also known as domain transformation, primarily used to transform low-resolution images to high resolution.
- **Cycle GAN:** It is used to perform image translation. E.g., we have trained it on a horse image dataset, and we can translate it into zebra images.
- **Info GAN** is the latest and advanced version of generative adversarial networks used for unsupervised machine learning.

References

1. https://en.wikipedia.org/wiki/Generative_adversarial_network

UNIT-V (Neural Network Architectures)
Spectral CNN
Lecture No.3

The term *"spectral"* may intimidate some machine learning practitioners (or bring back memories of quantum operators for physics students), but in simple terms, spectral theory refers to the study of the properties of linear operators, especially those pertaining to eigenvalues and eigenvectors.

In the context of graph theory, spectral theory focuses on examining the properties of the adjacency and Laplacian matrices associated with a graph. The adjacency matrix is more intuitive and easier to define. For a graph G = (V, E) with n nodes, we define the adjacency matrix A, whose entries are determined by,

$$a_{i,j} = \begin{cases} 1 & \text{if } \{i,j\} \in E \\ 0 & \text{otherwise} \end{cases}$$

We can provide a similar definition for the degree matrix D, which is a diagonal matrix that encompasses all the information regarding the local connectivity of a node.

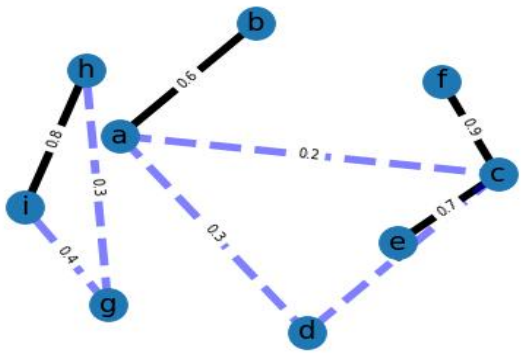
$$d_{i,j} = \begin{cases} d(i) & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

where d(i) is the number of neighbors of this node.

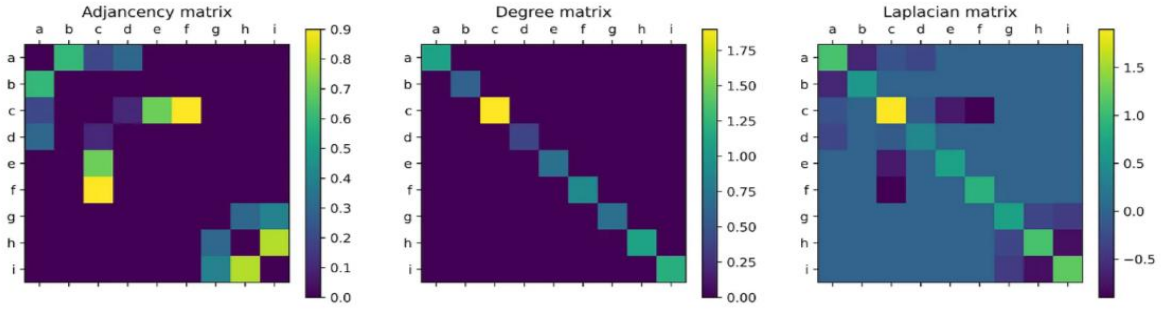
On the other hand, we can obtain our initial definition of the Laplacian matrix L by employing a similar approach, defining each entry as

$$l_{i,j} = \begin{cases} -1 & \text{if } \{i,j\} \in E \\ d(i) & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

We can simplify the element-wise definition by observing that the previous definition is equivalent to L = D — A.



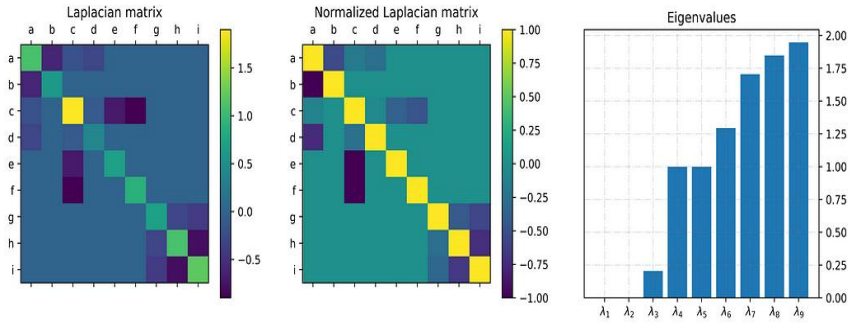
Spectral Graph



The image displays matrices associated with the graph. It is important to note that the values within the eigenvectors do not adhere strictly to specific minimum and maximum bounds, as they vary based on the graph's characteristics and structure.

In many cases, certain nodes exhibit a higher number of connections compared to others. This disparity in node degrees can impact the performance of our algorithms. To address this issue, we employ a normalization technique based on random walks. This technique assigns weights to the edges based on the node degrees, thereby balancing the influence of highly connected nodes, and ensuring that nodes with lower degrees are not overshadowed. This is achieved by dividing each entry by the sum of the degrees of the corresponding node.

$$\mathcal{L} = D^{-1}(D - A) = I - D^{-1}A$$



The image illustrates the normalized Laplacian matrix of the graph along with its corresponding eigenvalues. Notably, the eigenvector entries exhibit clear bounds ranging from -1 to 1. The presence of two components is reflected by the first two eigenvalues being equal to 0. Additionally, the maximum eigenvalue is observed to be approximately equal to two.

For now, let's shift our focus to the eigen decomposition of the matrix L. Eigen decomposition is a fundamental concept that enables us to break down a square matrix into a set of eigenvalues and eigenvectors. By expressing L as the product of three matrices,

$$\mathcal{L} = U\Lambda U^{-1} = U\Lambda U^{\intercal}$$

where the matrix U contains the eigenvectors of L, and the inner matrix is a diagonal matrix with the eigenvalues as its entries.

$$\Lambda = \begin{bmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n \end{bmatrix}$$

However, we encounter a problem: the eigenvalues of a matrix can be complex numbers. Fortunately, this is not the case for the Laplacian matrix. To understand why, we introduce a new matrix that solely considers the Laplacian of every edge {u, v} in E.

$$L_{G_{\{u,v\}}}(i,j) = \begin{cases} 1 & \text{if } i = j \text{ and } i \in \{u,v\} \\ -1 & \text{if } i = u \text{ and } j = v, \text{ or vice versa} \\ 0 & \text{otherwise} \end{cases}$$

Using these matrices, we can provide a new definition of the Laplacian matrix as the sum of each of the aforementioned matrices.

$$L_G = \sum_{\{u,v\} \in E} L_{G_{\{u,v\}}}$$

This new definition provides us with additional information about the Laplacian matrix. For instance, it allows us to establish the Laplacian quadratic form

$$\vec{x}^{\intercal} L_{G_{\{u,v\}}} \vec{x} = (x_u - x_v)^2 \text{ for all } \vec{x} \in \mathbb{R}^n$$

and with that in mind, we can easily show that L is self-adjoint. As a result, we get some interesting properties.

- L has n linear independent eigenvectors.
- The eigenvectors correspond to different eigenvalues of L and are orthogonal to each other.
- All eigenvalues are real and non-negative.

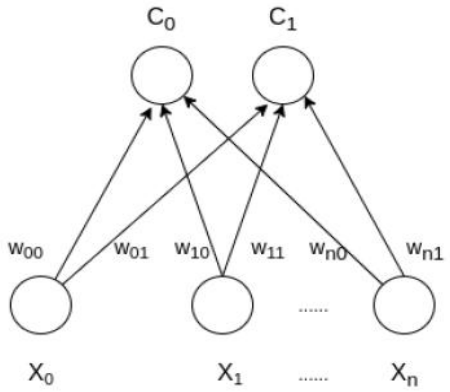
The eigenvalues of L provide us with even more valuable information than expected. For instance, if we sort and relabel the eigenvalues such that λ_1 corresponds to the smallest eigenvalue, we observe that this value is always zero. This occurrence is not coincidental; the multiplicity of zero as an eigenvalue indicates the number of connected components in the graph. Specifically, if λ_2 is greater than zero, then our graph is connected. On the other hand, the multiplicity of two as an eigenvalue corresponds to the number of bipartite connected components in our graph with at least two vertices.

References

1. https://www.sciencedirect.com/science/article/abs/pii/S0168169921006414

UNIT-V (Neural Network Architectures)
Self-Organizing Maps
Lecture No.4

Self Organizing Map (or Kohonen Map or SOM) is a type of Artificial Neural Network which is also inspired by biological models of neural systems from the 1970s. It follows an unsupervised learning approach and trained its network through a competitive learning algorithm. SOM is used for clustering and mapping (or dimensionality reduction) techniques to map multidimensional data onto lower-dimensional which allows people to reduce complex problems for easy interpretation. SOM has two layers, one is the Input layer and the other one is the Output layer. The architecture of the Self Organizing Map with two clusters and n input features of any sample is given below:



SOM

How do SOM works?

Let's say an input data of size (m, n) where m is the number of training examples and n is the number of features in each example. First, it initializes the weights of size (n, C) where C is the number of clusters. Then iterating over the input data, for each training example, it updates the winning vector (weight vector with the shortest distance (e.g Euclidean distance) from training example). Weight updation rule is given by :

$$w_{ij} = w_{ij}(old) + alpha(t) * (x_i^k - w_{ij}(old))$$

where alpha is a learning rate at time t, j denotes the winning vector, i denotes the ith feature of training example and k denotes the kth training example from the input data. After training the SOM network, trained weights are used for clustering new examples. A new example falls in the cluster of winning vectors.

Algorithm

Training:

Step 1: Initialize the weights w_{ij} random value may be assumed. Initialize the learning rate α.

Step 2: Calculate squared Euclidean distance.

D(j) = Σ (wij – xi)^2 where i=1 to n and j=1 to m

Step 3: Find index J, when D(j) is minimum that will be considered as winning index.

Step 4: For each j within a specific neighborhood of j and for all i, calculate the new weight.

wij(new)=wij(old) + α[xi – wij(old)]

Step 5: Update the learning rule by using :

α(t+1) = 0.5 * t

Step 6: Test the Stopping Condition.

References

1. https://www.geeksforgeeks.org/self-organising-maps-kohonen-maps/

UNIT-V (Neural Network Architectures)
Restricted Boltzmann Machines
Lecture No.5

Restricted Boltzmann Machine (RBM) is a type of artificial neural network that is used for unsupervised learning. It is a type of generative model that is capable of learning a probability distribution over a set of input data.RBM was introduced in the mid-2000s by Hinton and Salakhutdinov as a way to address the problem of unsupervised learning. It is a type of neural network that consists of two layers of neurons – a visible layer and a hidden layer. The visible layer represents the input data, while the hidden layer represents a set of features that are learned by the network.

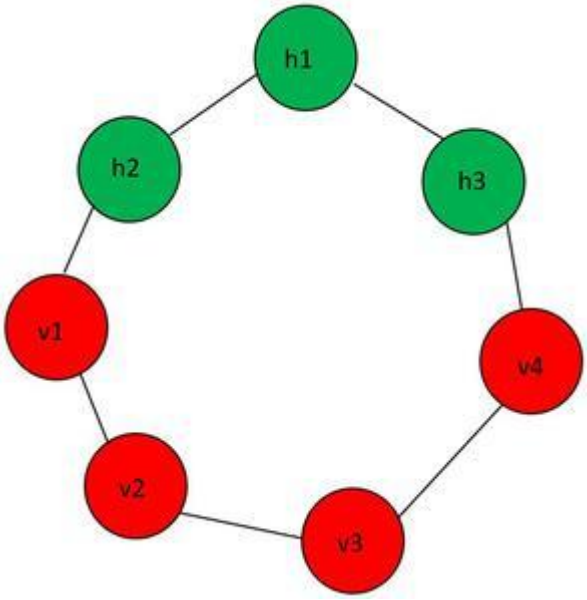
The RBM is called "restricted" because the connections between the neurons in the same layer are not allowed. In other words, each neuron in the visible layer is only connected to neurons in the hidden layer, and vice versa. This allows the RBM to learn a compressed representation of the input data by reducing the dimensionality of the input.The RBM is trained using a process called contrastive divergence, which is a variant of the stochastic gradient descent algorithm. During training, the network adjusts the weights of the connections between the neurons in order to maximize the likelihood of the training data. Once the RBM is trained, it can be used to generate new samples from the learned probability distribution.

RBM has found applications in a wide range of fields, including computer vision, natural language processing, and speech recognition. It has also been used in combination with other neural network architectures, such as deep belief networks and deep neural networks, to improve their performance.

What are Boltzmann Machines?

It is a network of neurons in which all the neurons are connected to each other. In this machine, there are two layers named visible layer or input layer and hidden layer. The visible layer is denoted as **v** and the hidden layer is denoted as the **h.** In Boltzmann machine, there is no output layer. Boltzmann machines are random and generative neural networks capable of learning internal representations and are able to represent and (given enough time) solve tough combinatoric problems.

The Boltzmann distribution (also known as **Gibbs Distribution**) which is an integral part of Statistical Mechanics and also explain the impact of parameters like Entropy and Temperature on the Quantum States in Thermodynamics. Due to this, it is also known as **Energy-Based Models (EBM)**. It was invented in 1985 by Geoffrey Hinton, then a Professor at Carnegie Mellon University, and Terry Sejnowski, then a Professor at Johns Hopkins University



RBM Graph(Energy Based Model)

What is Restricted Boltzmann Machines (RBM)?

A restricted term refers to that we are not allowed to connect the same type layer to each other. In other words, the two neurons of the input layer or hidden layer can't connect to each other. Although the hidden layer and visible layer can be connected to each other. As in this machine, there is no output layer so the question arises how we are going to identify, adjust the weights and how to measure that our prediction is accurate or not. All the questions have one answer that is Restricted Boltzmann Machine.

The RBM algorithm was proposed by Geoffrey Hinton (2007), which learns probability distribution over its sample training data inputs. It has seen wide applications in different areas of supervised/unsupervised machine learning such as feature learning, dimensionality reduction, classification, collaborative filtering, and topic modeling.

Consider the example movie rating discussed in the recommender system section.
Movies like Avengers, Avatar, and Interstellar have strong associations with the latest fantasy and science fiction factor. Based on the user rating RBM will discover latent

factors that can explain the activation of movie choices. In short, RBM describes variability among correlated variables of input dataset in terms of a potentially lower number of unobserved variables.

Applications of Restricted Boltzmann Machine

Restricted Boltzmann Machines (RBMs) have found numerous applications in various fields, some of which are:

- **Collaborative filtering:** RBMs are widely used in collaborative filtering for recommender systems. They learn to predict user preferences based on their past behavior and recommend items that are likely to be of interest to the user.
- **Image and video processing:** RBMs can be used for image and video processing tasks such as object recognition, image denoising, and image reconstruction. They can also be used for tasks such as video segmentation and tracking.
- **Natural language processing:** RBMs can be used for natural language processing tasks such as language modeling, text classification, and sentiment analysis. They can also be used for tasks such as speech recognition and speech synthesis.
- **Bioinformatics:** RBMs have found applications in bioinformatics for tasks such as protein structure prediction, gene expression analysis, and drug discovery.
- **Financial modeling:** RBMs can be used for financial modeling tasks such as predicting stock prices, risk analysis, and portfolio optimization.
- **Anomaly detection:** RBMs can be used for anomaly detection tasks such as fraud detection in financial transactions, network intrusion detection, and medical diagnosis.
- It is used in Filtering.
- It is used in Feature Learning.
- It is used in Classification.
- It is used in Risk Detection.
- It is used in Business and Economic analysis.

How do Restricted Boltzmann Machines work?

In RBM there are two phases through which the entire RBM works:

1st Phase: In this phase, we take the input layer and using the concept of weights and biased we are going to activate the hidden layer. This process is said to be Feed Forward Pass. In Feed Forward Pass we are identifying the positive association and negative association.

Feed Forward Equation:

- **Positive Association** — When the association between the visible unit and the hidden unit is positive.
- **Negative Association** — When the association between the visible unit and the hidden unit is negative.

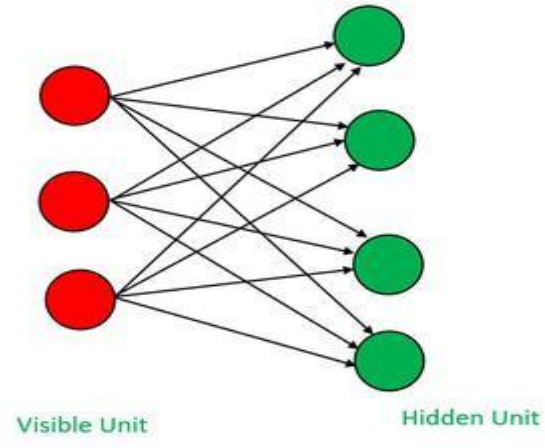
2nd Phase: As we don't have any output layer. Instead of calculating the output layer, we are reconstructing the input layer through the activated hidden state. This process is said to be Feed Backward Pass. We are just backtracking the input layer through the activated hidden neurons. After performing this we have reconstructed Input through the activated hidden state. So, we can calculate the error and adjust weight in this way:

Feed Backward Equation:

- **Error =** Reconstructed Input Layer-Actual Input layer
- **Adjust Weight =** Input*error*learning rate (0.1)

After doing all the steps we get the pattern that is responsible to activate the hidden neurons. To understand how it works:

Let us consider an example in which we have some assumption that V1 visible unit activates the h1 and h2 hidden unit and V2 visible unit activates the h2 and h3 hidden. Now when any new visible unit let V5 has come into the machine and it also activates the h1 and h2 unit. So, we can back trace the hidden units easily and also identify that the characteristics of the new V5 neuron is matching with that of V1. This is because V1 also activated the same hidden unit earlier.



Restricted Boltzmann Machines

Types of RBM:

There are mainly two types of Restricted Boltzmann Machine (RBM) based on the types of variables they use:

1. **Binary RBM:** In a binary RBM, the input and hidden units are binary variables. Binary RBMs are often used in modeling binary data such as images or text.
2. **Gaussian RBM:** In a Gaussian RBM, the input and hidden units are continuous variables that follow a Gaussian distribution. Gaussian RBMs are often used in modeling continuous data such as audio signals or sensor data.

Apart from these two types, there are also variations of RBMs such as:

1. **Deep Belief Network (DBN):** A DBN is a type of generative model that consists of multiple layers of RBMs. DBNs are often used in modeling high-dimensional data such as images or videos.
2. **Convolutional RBM (CRBM):** A CRBM is a type of RBM that is designed specifically for processing images or other grid-like structures. In a CRBM, the connections between the input and hidden units are local and shared, which makes it possible to capture spatial relationships between the input units.
3. **Temporal RBM (TRBM):** A TRBM is a type of RBM that is designed for processing temporal data such as time series or video frames. In a TRBM, the hidden units are connected across time steps, which allows the network to model temporal dependencies in the data.

References

1. https://en.wikipedia.org/wiki/Restricted_Boltzmann_machine

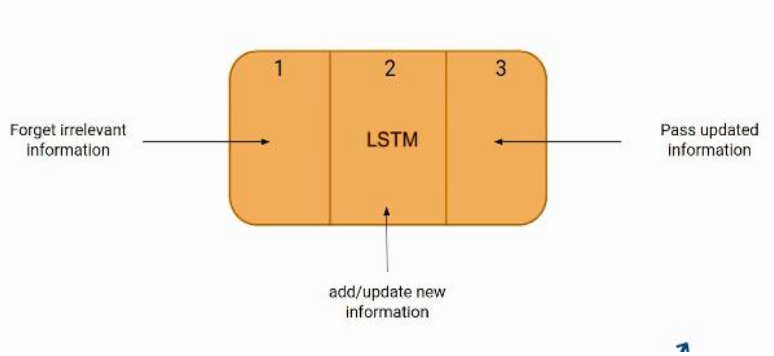
UNIT-V (Neural Network Architectures)
Long Term Memory Networks
Lecture No.6

LSTM (Long Short-Term Memory) is a recurrent neural network (RNN) architecture widely used in Deep Learning. It excels at capturing long-term dependencies, making it ideal for sequence prediction tasks.

Unlike traditional neural networks, LSTM incorporates feedback connections, allowing it to process entire sequences of data, not just individual data points. This makes it highly effective in understanding and predicting patterns in sequential data like time series, text, and speech.LSTM has become a powerful tool in artificial intelligence and deep learning, enabling breakthroughs in various fields by uncovering valuable insights from sequential data.

LSTM Architecture

In the introduction to long short-term memory, we learned that it resolves the vanishing gradient problem faced by RNN, so now, in this section, we will see how it resolves this problem by learning the architecture of the LSTM. At a high level, LSTM works very much like an RNN cell. Here is the internal functioning of the LSTM network. The LSTM network architecture consists of three parts, as shown in the image below, and each part performs an individual function.



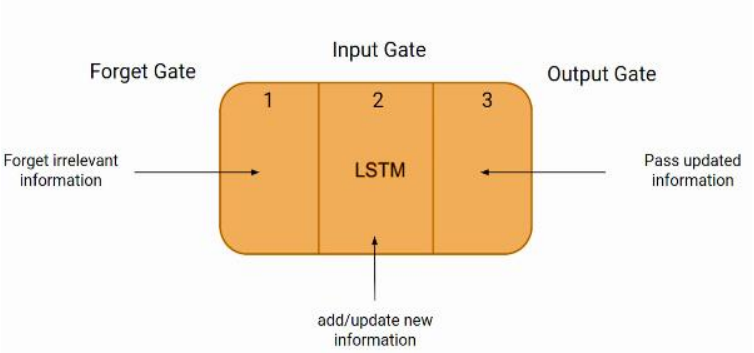
LSTM Architecture

The Logic Behind LSTM

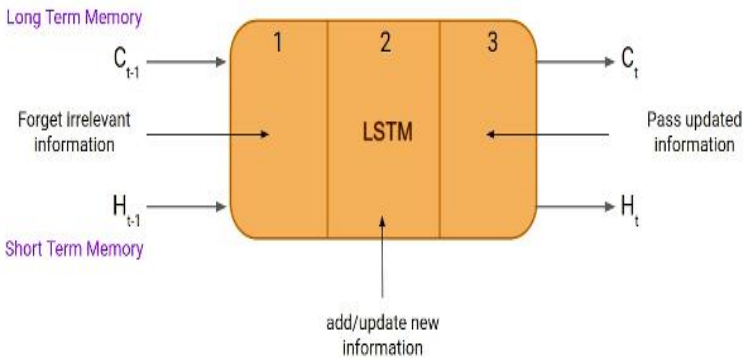
The first part chooses whether the information coming from the previous timestamp is to be remembered or is irrelevant and can be forgotten. In the second part, the cell tries to learn new information from the input to this cell. At last, in the third part, the

cell passes the updated information from the current timestamp to the next timestamp. This one cycle of LSTM is considered a single-time step.

These three parts of an LSTM unit are known as gates. They control the flow of information in and out of the memory cell or lstm cell. The first gate is called **Forget gate, the** second gate is known as **the Input gate,** and the last one is **the Output gate**. An LSTM unit that consists of these three gates and a memory cell or lstm cell can be considered as a layer of neurons in traditional feedforward neural network, with each neuron having a hidden layer and a current state.



Just like a simple RNN, an LSTM also has a hidden state where H(t-1) represents the hidden state of the previous timestamp and Ht is the hidden state of the current timestamp. In addition to that, LSTM also has a cell state represented by C(t-1) and C(t) for the previous and current timestamps, respectively. Here the hidden state is known as Short term memory, and the cell state is known as Long term memory. Refer to the following image.



It is interesting to note that the cell state carries the information along with all the timestamps.



LSTM

Bob is a nice person. Dan on the other hand is evil.

Example of LTSM Working

Let's take an example to understand how LSTM works. Here we have two sentences separated by a full stop. The first sentence is "Bob is a nice person," and the second sentence is "Dan, on the Other hand, is evil". It is very clear, in the first sentence, we are talking about Bob, and as soon as we encounter the full stop(.), we started talking about Dan.

As we move from the first sentence to the second sentence, our network should realize that we are no more talking about Bob. Now our subject is Dan. Here, the Forget gate of the network allows it to forget about it. Let's understand the roles played by these gates in LSTM architecture.

Forget Gate

In a cell of the LSTM neural network, the first step is to decide whether we should keep the information from the previous time step or forget it. Here is the equation for forget gate.

Forget Gate:

- $f_t = \sigma (x_t * U_f + H_{t-1} * W_f)$

Let's try to understand the equation, here

- Xt: input to the current timestamp.
- Uf: weight associated with the input
- Ht-1: The hidden state of the previous timestamp
- Wf: It is the weight matrix associated with the hidden state

Later, a sigmoid function is applied to it. That will make ft a number between 0 and 1. This ft is later multiplied with the cell state of the previous timestamp, as shown below.

$C_{t-1} * f_t = 0$ ...if $f_t = 0$ (forget everything)

$C_{t-1} * f_t = C_{t-1}$ ...if $f_t = 1$ (forget nothing)

Input Gate

Let's take another example.

"Bob knows swimming. He told me over the phone that he had served the navy for four long years."

So, in both these sentences, we are talking about Bob. However, both give different kinds of information about Bob. In the first sentence, we get the information that he knows swimming. Whereas the second sentence tells, he uses the phone and served in the navy for four years.

Now just think about it, based on the context given in the first sentence, which information in the second sentence is critical? First, he used the phone to tell, or he served in the navy. In this context, it doesn't matter whether he used the phone or any other medium of communication to pass on the information. The fact that he was in the navy is important information, and this is something we want our model to remember for future computation. This is the task of the Input gate.

The input gate is used to quantify the importance of the new information carried by the input. Here is the equation of the input gate

Input Gate:

- $i_t = \sigma (x_t * U_i + H_{t-1} * W_i)$

Here,

- Xt: Input at the current timestamp t
- Ui: weight matrix of input
- Ht-1: A hidden state at the previous timestamp
- Wi: Weight matrix of input associated with hidden state

Again we have applied the sigmoid function over it. As a result, the value of I at timestamp t will be between 0 and 1.

New Information

- $N_t = \tanh(x_t * U_c + H_{t-1} * W_c)$ (new information)

Now the new information that needed to be passed to the cell state is a function of a hidden state at the previous timestamp t-1 and input x at timestamp t. The activation function here is tanh. Due to the tanh function, the value of new information will be between -1 and 1. If the value of Nt is negative, the information is subtracted from the cell state, and if the value is positive, the information is added to the cell state at the current timestamp.

However, the Nt won't be added directly to the cell state. Here comes the updated equation:

$$C_t = f_t * C_{t-1} + i_t * N_t \text{ (updating cell state)}$$

Here, Ct-1 is the cell state at the current timestamp, and the others are the values we have calculated previously.

Output Gate

Now consider this sentence.

"Bob single-handedly fought the enemy and died for his country. For his contributions, brave_____."

During this task, we have to complete the second sentence. Now, the minute we see the word brave, we know that we are talking about a person. In the sentence, only Bob is brave, we cannot say the enemy is brave, or the country is brave. So based on the current expectation, we have to give a relevant word to fill in the blank. That word is our output, and this is the function of our Output gate.

Here is the equation of the Output gate, which is pretty similar to the two previous gates.

Output Gate:

- $o_t = \sigma(x_t * U_o + H_{t-1} * W_o)$

Its value will also lie between 0 and 1 because of this sigmoid function. Now to calculate the current hidden state, we will use Ot and tanh of the updated cell state. As shown below.

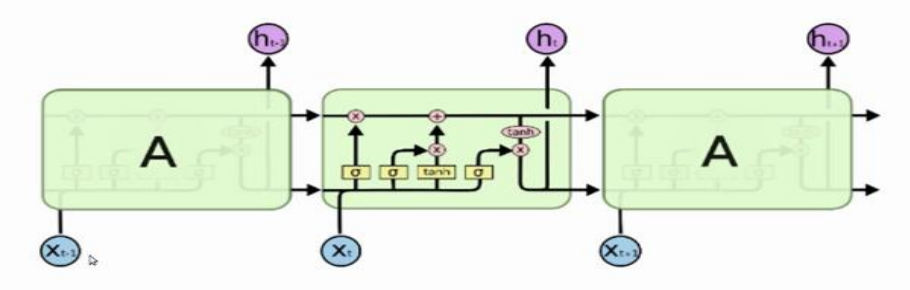
$$H_t = o_t * \tanh(C_t)$$

It turns out that the hidden state is a function of Long term memory (Ct) and the current output. If you need to take the output of the current timestamp, just apply the SoftMax activation on hidden state Ht.

$$\text{Output} = \text{Softmax}(H_t)$$

Here the token with the maximum score in the output is the prediction.

This is the More intuitive diagram of the LSTM network.



LSTM Working Process

What are Bidirectional LSTMs?

Bidirectional LSTMs (Long Short-Term Memory) are a type of recurrent neural network (RNN) architecture that processes input data in both forward and backward directions. In a traditional LSTM, the information flows only from past to future, making predictions based on the preceding context. However, in bidirectional LSTMs, the network also considers future context, enabling it to capture dependencies in both directions.

The bidirectional LSTM comprises two LSTM layers, one processing the input sequence in the forward direction and the other in the backward direction. This allows the network to access information from past and future time steps simultaneously. As a result, bidirectional LSTMs are particularly useful for tasks that require a comprehensive understanding of the input sequence, such as natural language processing tasks like sentiment analysis, machine translation, and named entity recognition.By incorporating information from both directions, bidirectional LSTMs enhance the model's ability to capture long-term dependencies and make more accurate predictions in complex sequential data.

References

1. https://www.geeksforgeeks.org/deep-learning-introduction-to-long-short-term-memory/

UNIT-V (Neural Network Architectures)
Deep Reinforcement Learning
Lecture No.7

In the past few years, Deep Learning techniques have become very popular. Deep Reinforcement Learning is the combination of Reinforcement Learning with Deep Learning techniques to solve challenging sequential decision-making problems.
The use of deep learning is most useful in problems with high-dimensional state space. This means that with deep learning, Reinforcement Learning is able to solve more complicated tasks with lower prior knowledge because of its ability to learn different levels of abstractions from data.

To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. This makes it possible for machines to mimic some human problem-solving capabilities, even in high-dimensional space, which only a few years ago was difficult to conceive.

Applications of Deep Reinforcement Learning

Some prominent projects used deep Reinforcement Learning in games with results that are far beyond what is humanly possible. Deep RL techniques have demonstrated their ability to tackle a wide range of problems that were previously unsolved.

Deep RL has achieved human-level or superhuman performance for many two-player or even multi-player games. Such achievements with popular games are significant because they show the potential of deep Reinforcement Learning in a variety of complex and diverse tasks that are based on high-dimensional inputs. With games, we have good or even perfect simulators, and can easily generate unlimited data.

- **Atari 2600 games:** Machines achieved superhuman-level performance in playing Atari games.
- **Go:** Mastering the game of Go with deep neural networks.
- **Poker:** AI is able to beat professional poker players in the game of heads-up no-limit Texas hold'em.

- **Quake III:** An agent achieved human-level performance in a 3D multiplayer first-person video game, using only pixels and game points as input.
- **Dota 2:** An AI agent learned to play Dota 2 by playing over 10,000 years of games against itself (OpenAI Five).
- **StarCraft II:** An agent was able to learn how to play StarCraft II a 99\% win-rate, using only 1.08 hours on a single commercial machine.

Those achievements set the basis for the development of real-world deep reinforcement learning applications:

- **Robot control:** Robotics is a classical application area for reinforcement learning. Robust adversarial reinforcement learning is applied as an agent operates in the presence of a destabilizing adversary that applies disturbance forces to the system. The machine is trained to learn an optimal destabilization policy. AI-powered robots have a wide range of applications, e.g. in manufacturing, supply chain automation, healthcare, and many more.
- **Self-driving cars:** Deep Reinforcement Learning is prominently used with autonomous driving. Autonomous driving scenarios involve interacting agents and require negotiation and dynamic decision-making, which suits Reinforcement Learning.
- **Healthcare:** In the medical field, Artificial Intelligence (AI) has enabled the development of advanced intelligent systems able to learn about clinical treatments, provide clinical decision support, and discover new medical knowledge from the huge amount of data collected. Reinforcement Learning enabled advances such as personalized medicine that is used to systematically optimize patient health care, in particular, for chronic conditions and cancers, using individual patient information.
- **Other:** In terms of applications, many areas are likely to be impacted by the possibilities brought by deep Reinforcement Learning, such as finance, business management, marketing, resource management, education, smart grids, transportation, science, engineering, or art. In fact, Deep RL systems are already in production environments. For example, Facebook uses Deep Reinforcement Learning for pushing notifications and for faster video loading with smart prefetching.

Challenges of Deep Reinforcement Learning

Multiple challenges arise in applying Deep Reinforcement Learning algorithms. In general, it is difficult to explore the environment efficiently or to generalize good behavior in a slightly different context. Therefore, multiple algorithms have been proposed for the Deep

Reinforcement Learning framework, depending on a variety of settings of the sequential decision-making tasks.

Many challenges appear when moving from a simulated setting to solving real-world problems.

- **Limited freedom of the agent:** In practice, even in the case where the task is well-defined (with explicit reward functions), a fundamental difficulty lies in the fact that it is often not possible to let the agent interact freely and sufficiently in the actual environment, due to safety, cost or time constraints.
- **Reality gap:** There may be situations, where the agent is not able to interact with the true environment but only with an inaccurate simulation of it. The reality gap describes the difference between the learning simulation and the effective real-world domain.
- **Limited observations:** For some cases, the acquisition of new observations may not be possible anymore (e.g., the batch setting). Such scenarios occur, for example, in medical trials or tasks that depend on weather conditions or trading markets such as stock markets.

How those challenges can be addressed?

- **Simulation:** For many cases, a solution is the development of a simulator that is as accurate as possible.
- **Algorithm Design:** The design of the learning algorithms and their level of generalization have a great impact.
- **Transfer Learning:** Transfer learning is a crucial technique to utilize external expertise from other tasks to benefit the learning process of the target task.

References

1. https://www.analyticsvidhya.com/blog/2021/03/introduction-to-long-short-term-memory-lstm/

UNIT-V (Neural Network Architectures)
Tensor Flow, Keras or MatConNet for implementation
Lecture No.8 & 9

TensorFlow
TensorFlow is an open-source platform for machine learning and a symbolic math library that is used for machine learning applications.

Advantages of TensorFlow
Tensor flow has a better graph representation for a given data rather than any other top platform out there.

- Tensor flow has the advantage that it does support and uses many backend software like GUI and ASIC.
- When it comes to community support tensor flow has the best.
- Tensor flow also helps in debugging the sub-part of the graphs.
- Tensor flow has shown a better performance when compared with other platforms.
- Easy to extend as it gives freedom to add custom blocks to build on new ideas.

Disadvantages of TensorFlow
- Tensor flow not specifically designed for the Windows operating systems but it is designed for other OS like Linux but tensor flow can be installed in windows with the help of a python package installer(pip).
- The speed of the tensor flow is less when it is compared to other platforms of the same type.
- For a better understanding of tensor flow, the user must have the fundamentals of calculus.
- Tensor flow does not support OpenCL.

Keras
It is an Open Source Neural Network library that runs on top of Theano or Tensorflow. It is designed to be fast and easy for the user to use. It is a useful library to construct any deep learning algorithm of whatever choice we want.

Advantages of Keras
- Keras is the best platform out there to work on neural network models.
- The API that Keras has a user-friendly where a beginner can easily understand.
- Keras has the advantage that it can choose any libraries which support it for its backend support.

- Keras provides various pre-trained models which help the user in further improving the models the user is designing.
- When it comes to community support Keras has the best like stack overflow.

Disadvantages of Keras

- The major drawback of Keras is it is a low-level application programming interface.
- Few of the pre-trained models that the Keras has been not much supportive when it comes to designing of some models.
- The errors given by the Keras library were not much helpful for the user.

References

1. https://www.tensorflow.org/guide/keras

2. Deep Learning Step by Step with Python, N D Lewis, 2016